

Assignment 1 — From FPGA (PL) to CPU (PS)

Name: Ioannis Tarasidis

1. Introduction to the task

The goal of this assignment was to design and demonstrate a message path across the programmable logic (PL) and processing system (PS) on a PYNQ-Z2 board. Specifically, the PL stores a message in on-chip block RAM (BRAM), and the PS retrieves and prints the message through its UART. The design leverages an AXI interconnect and an AXI BRAM controller to expose the BRAM to the PS.

First, we created a Vivado Block Design that integrates the ZYNQ7 Processing System, an AXI Interconnect, an AXI BRAM Controller with a Block Memory Generator, and the standard Processor System Reset logic. After validating the design and assigning base addresses in the Address Editor (with the BRAM mapped to 0x40000000), we generated output products, ran synthesis and implementation, and produced a bitstream. We exported the hardware to an XSA and switched to Vitis to build an FSBL and the bare-metal PS application. For bring-up and verification, we used XSCT to program the PL, run ps7_init and the FSBL to initialize, and add memory map entries so XSCT could read and write the PL BRAM cleanly. Finally, we opened the PS UART on the host, downloaded the application, and started producing messages, confirming that the PS printed the expected output.

Short workflow

1. Create Vivado Block Design: ps7 + AXI Interconnect + AXI BRAM Controller + Block Memory Generator (BRAM) + optional AXI UARTLite or PL I/O for external UART.
2. Assign addresses (BRAM at 0x4000_0000 in this implementation), generate bitstream and export .xsa.
3. Create Vitis platform and build a bare-metal PS app that polls BRAM and prints payload via PS UART.

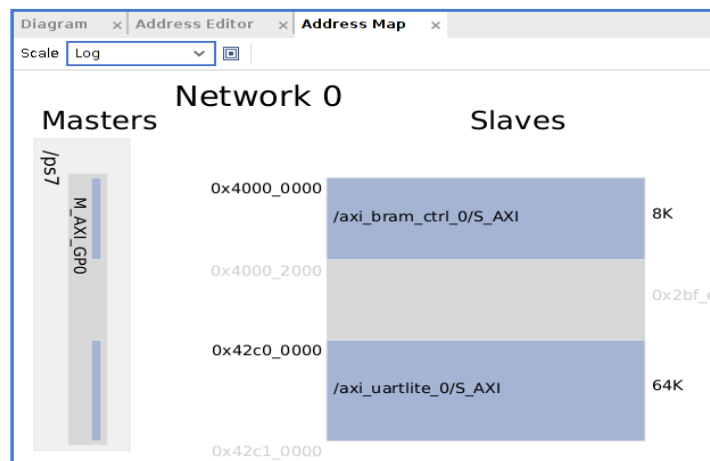


Fig 1: Address Map of the Block Design

4. Board bring-up (XSCT): program PL, initialize PS/DDR (FSBL), enable level shifters / release fabric reset, run app.

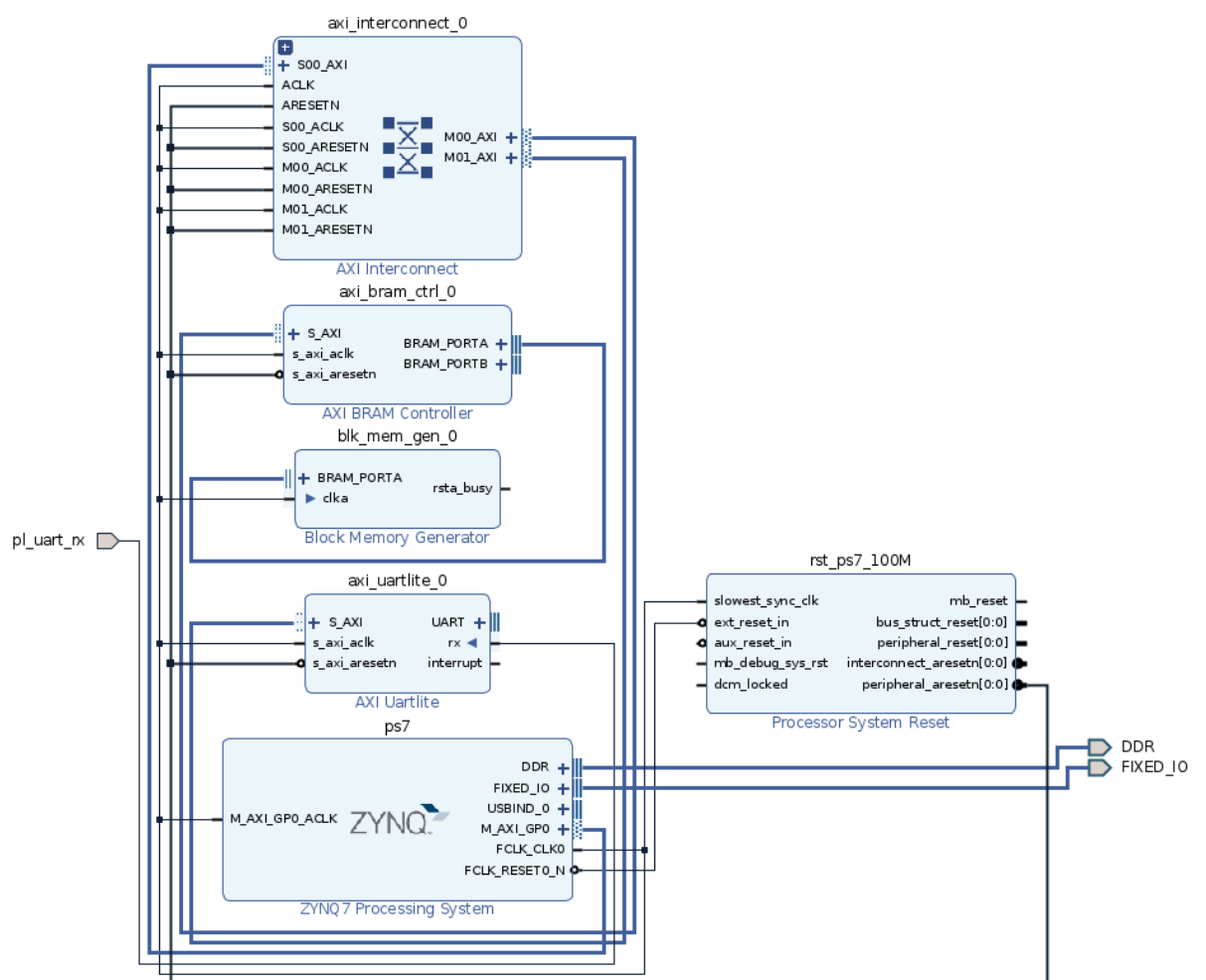


Fig 2: The Complete Block Design

2. Explanation of components

At the heart of the hardware is the ZYNQ7 Processing System (ps7), which provides the dual Cortex-A9 CPUs and PS peripherals and drives clocks and resets into the PL. The ps7 exposes the M_AXI_GP0 master port, which allows software running on the PS to issue AXI transactions into the PL address space. In our design, this master port connects through an AXI Interconnect to the AXI BRAM Controller. The interconnect controls access and provides the pathway by which the PS reaches the memory mapped BRAM window.

The AXI BRAM Controller, paired with a Block Memory Generator, implements a small on-chip memory inside the PL that is accessible from the PS. We mapped this memory at base address 0x40000000 with an 8 KiB window. We also defined arbitrarily the first 32-bit word at this base address as the “length” indicating the number of valid payload bytes that follow. This allows a producer in the PL to publish a message and a consumer in the PS to detect and read it with minimal logic.

The design optionally includes an AXI UARTLite core that offers a memory mapped UART inside the PL. Although UARTLite was present in the block design, in the demonstration we relied only on the external UMFT234XD USB to UART module connected to a receive signal (pl_uart_rx) as the producer input path. The Processor System Reset block ensures that all AXI-attached peripherals in the PL are held in reset and then released in a controlled, clock-synchronous manner derived from the pynq’s FCLK.

3. Testing

We validated the ps7 and bring-up sequence by following a consistent XSCT procedure. After connecting to the hardware server, we sourced and executed ps7_init and downloaded the FSBL, then continued execution so the FSBL could initialize DDR and essential PS subsystems. We programmed the PL with the generated bitstream and enabled the PS↔PL level shifters. To make XSCT aware of the PL memory window and suppress mapping warnings, we added a memmap entry for 0x40000000 on the APU target. With this in place, a simple mrd from 0x40000000 confirmed that the PS could read the BRAM.

The AXI Interconnect required no explicit testing beyond observing that BRAM reads and writes completed without AXI decode or timeout errors. Successful mrd/mwr and the PS application’s ability to poll payload from the BRAM provided strong evidence that interconnect routing and clocking were correct. The AXI BRAM subsystem was exercised directly using XSCT by writing values and reading them back. The Processor System Reset block’s role was verified indirectly, in that PL peripherals behaved normally after the fabric reset was released via the register write and we did not observe any weird symptoms.

We did not run a dedicated simulation campaign for this submission except some basic sanity checks and a few tiny testbenches for simple operations (like testing if PS actually sends messages), choosing to focus the available time on testing and validation of the implemented design. A formal testbench could nevertheless replicate the observed behavior by writing a payload into the BRAM model and observing a reader model that consumes it, but these artifacts are not included here. On hardware, we tested the full system end-to-end. We opened the PS UART console on the host using the Xilinx TUL serial interface (trying /dev/ttyUSB1 and falling back to /dev/ttyUSB2 as needed). In XSCT we connected to the target, executed ps7_init, downloaded and ran the FSBL, and programmed the PL bitstream. We invoked ps7_post_config and then enabled the PS↔PL level shifters. We mapped the PL BRAM window for XSCT and performed a sanity read at 0x40000000. We then downloaded and ran the application ELF and looked for any fail messages. Finally, after initializing the application we observed the PS UART printing the expected messages.

```
xsct% mrd 0x40000000 4
40000000: 00000008
40000004: 00000008
40000008: 00000008
4000000C: 00000008

xsct% fpga -f {/home/john/Desktop/Embedded Systems Projects/Assignment 1/project_1/project_1.runs/impl_1/pl_ps_bd_wrapper.bit}
100% 3MB 1.8MB/s 00:02
xsct% fpga -state
FPGA is configured
```

Fig 3: Tests of the BRAM contents and the status of the FPGA after setting up the configuration respectively

4. Discussion

The results confirm that the BRAM-mediated handoff from PL to PS functions as intended. Once the board is brought up in the proper sequence and the PS↔PL level shifters are enabled, the PS can reliably access the PL BRAM window and the application can read messages. There are parts of the design that were not ultimately used (like the UARTLite block) but can be included in further improvements of this project, extending the functionality of the system. A large amount of time was spent dealing with memory access of the PYNQ board but the problems were ultimately solved by making XSCT aware of the PL memory window, but this pitfall can be blamed on inexperience.

```
Message received, length=8
```

Fig 4: The message printed in the receiver demonstrating the complete PS↔PL data path

5. Conclusion

In summary, we implemented and validated a ZYNQ PS↔PL data path in which the PL writes a message into BRAM at 0x40000000 and a bare-metal PS application reads the payload and prints it via the PS UART. The complete workflow from Vivado block design through Vitis builds to XSCT bring-up and interactive testing was executed and documented, and the hardware demonstration reliably reproduced the intended behavior. Through this task we gained significant insight into the inner mechanisms of a simple UART communication system and the common pitfalls in the implementation of such a project.

Citation: ChatGPT was used for improvements of the syntax of this report and in generation of code.

DEMO youtube link: <https://youtu.be/W6VeNONTLzI>

On the left side of the DEMO, all the commands run in the tests are shown in order, as well as explanations of their purpose. The upright terminal is the receiver where we aim to send the messages.